

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA



MANUAL TÉCNICO SISTEMA DE SOLICITUDES

Diego Iván Correa Navarrete
Juan Antonio González del Río
Diego Arturo Ramos Ávila
Ángel David Carlos Silva

LICENCIATURA EN INGENIERÍA DE SOFTWARE

Junio de 2026

Índice general

1. Introducción y propósito	2
2. Requisitos del sistema	3
2.1. Software requerido	3
2.2. Versiones del stack (provistas por los contenedores)	3
3. Arquitectura técnica	5
3.1. Estilo arquitectónico	5
3.2. Stack tecnológico	5
3.3. Aplicaciones Django	6
4. Estructura del proyecto	7
4.1. Árbol de la raíz del repositorio	7
4.2. Árbol de <code>app/</code>	7
4.3. Estructura interna de una app por capas	8
5. Instalación y despliegue	9
5.1. Requisitos previos	9
5.2. Instalación paso a paso	9
5.2.1. 1. Levantar el stack	9
5.2.2. 2. Aplicar migraciones	9
5.2.3. 3. Sembrar datos de desarrollo	9
5.2.4. 4. Acceder al sistema	10
5.3. URLs disponibles (tras levantar y sembrar)	10
5.4. Usuarios sembrados	10
5.5. Servicios del stack	10
6. Comandos de operación	12
7. Configuración	14
8. Pruebas	16
8.1. Pruebas unitarias y de integración	16
8.2. Cobertura	16
8.3. Pruebas de aceptación (behave + Selenium)	16
8.4. Pruebas de rendimiento (JMeter)	17
8.5. Complejidad ciclomática y mantenibilidad (radon)	17
8.6. Estilo (PEP 8) y tipado	18

9. Mantenimiento y extensión	19
9.1. Añadir una nueva característica respetando las capas	19
9.2. Datos sembrados (seeders)	19
9.3. Migraciones	20
10.Solución de problemas	21
10.1. Advertencia de certificado autofirmado	21
10.2. Puertos ocupados	21
10.3. Reconstruir la imagen	21
10.4. Estado inconsistente o base corrupta en desarrollo	21
10.5. Chromium ausente en pruebas de navegador	22

Capítulo 1

Introducción y propósito

El **Sistema de Solicitudes** de la Universidad Autónoma de Zacatecas (UAZ) es una aplicación web monolítica construida con Django que permite a alumnos y docentes presentar solicitudes académicas y administrativas mediante formularios dinámicos, dar seguimiento a su ciclo de vida con una máquina de estados, generar documentos PDF a partir de plantillas y gestionar todo el flujo con base en roles.

Este **Manual Técnico** está dirigido al personal de desarrollo, mantenimiento y operación del sistema. Su propósito es documentar, de forma verificable y reproducible:

- Los requisitos del entorno de ejecución y las versiones del stack.
- La arquitectura técnica y la organización del código.
- El procedimiento de instalación y despliegue con Docker.
- Los comandos de operación diaria.
- Las variables de configuración.
- La estrategia de pruebas (unitarias, integración, aceptación, rendimiento) y de aseguramiento de calidad.
- Las tareas de mantenimiento y extensión.
- La solución de los problemas más comunes.

Convenciones. Los identificadores de código (variables, funciones, clases, módulos) están en inglés; la copia visible para el usuario final está en español. A lo largo del manual, todo comando mostrado es ejecutable tal cual contra el repositorio; cada atajo de `make` se acompaña de su equivalente crudo de `docker compose`, de modo que no es indispensable tener `make` instalado.

Capítulo 2

Requisitos del sistema

El sistema se ejecuta íntegramente dentro de contenedores Docker, por lo que la máquina anfitriona requiere muy pocas dependencias instaladas de forma nativa.

2.1. Software requerido

Componente	Versión mínima	Notas
Docker Desktop / Docker Engine	Engine 24+	Incluye el motor de contenedores.
Docker Compose	v2	Se invoca como docker compose (subcomando, no el binario antiguo docker-compose).
git	Cualquier versión reciente	Para clonar el repositorio.
make	Opcional	Solo por conveniencia; cada objetivo tiene su equivalente crudo de docker compose .

2.2. Versiones del stack (provistas por los contenedores)

Las siguientes versiones quedan fijadas por las imágenes y los archivos de dependencias, no por la máquina anfitriona:

Tecnología	Versión
Python	3.13 (imagen <code>python:3.13.13-slim-bookworm</code>)
Django	5.1.15
Pydantic	2.13.3
PostgreSQL (desarrollo)	18.3
Nginx (TLS dev)	1.30.0-alpine
Mailhog	v1.0.1
Tailwind CSS (CLI standalone)	4.2.4

Tecnología	Versión
WeasyPrint	68.1
Gunicorn	25.3.0
psycpg (binary)	3.3.3
PyJWT	2.12.1

Recursos. Para desarrollo basta una máquina con Docker Desktop y unos 2 GB de RAM libres. Los puertos 80, 443, 5432 y 8025 del anfitrión deben estar disponibles (ver capítulo de Solución de problemas).

Capítulo 3

Arquitectura técnica

3.1. Estilo arquitectónico

El sistema sigue una arquitectura por capas **Vista** → **Servicio** → **Repositorio** (*View* → *Service* → *Repository*), con DTOs de Pydantic v2 en cada frontera entre capas:

- **Vista (View)**. Recibe la petición HTTP, valida la entrada con formularios de Django y delega la lógica al servicio. Renderiza plantillas del lado del servidor a partir de DTOs de Pydantic (no hay DRF ni API JSON por defecto).
- **Servicio (Service)**. Contiene la lógica de negocio y las reglas de transición de estado. No conoce HTTP ni el ORM; opera sobre DTOs.
- **Repositorio (Repository)**. Único lugar donde vive el ORM de Django. Traduce entre modelos del ORM y DTOs.

Todas las excepciones heredan de `_shared.exceptions.AppError` y un *middleware* las mapea a respuestas HTTP. La validación de JWT se realiza también en *middleware* (el proveedor de identidad es externo).

3.2. Stack tecnológico

- **Python 3.13** + **Django 5.x** con plantillas del lado del servidor (sin DRF).
- **Pydantic v2** para los DTOs en cada frontera de capa.
- **PostgreSQL** en producción y desarrollo; **SQLite** para la suite de pruebas en proceso. El ORM queda contenido en los repositorios.
- **Tailwind CSS v4** + **Alpine.js v3** + iconos **Lucide** para la interfaz. El binario standalone de Tailwind se ejecuta dentro del contenedor **web** (no requiere Node).
- **WeasyPrint** para la generación de PDF a partir de plantillas HTML/CSS.
- **Validación de JWT** en *middleware* (proveedor de autenticación externo).
- **Celery** + **Redis** para correo asíncrono (opcional).
- **pytest** + **pytest-django**, con `model_bakery`, `freezegun` y `responses` para las pruebas.

3.3. Aplicaciones Django

El proyecto se organiza en una aplicación Django por dominio:

App	Responsabilidad
<code>_shared</code>	Infraestructura transversal: excepciones (<code>AppError</code>), <i>middleware</i> , auditoría y utilidades comunes.
<code>usuarios</code>	Validación de JWT, roles, perfil y directorio de usuarios. El proveedor de identidad es externo (SIGA).
<code>solicitudes</code>	Núcleo del sistema: tipos de solicitud, formularios dinámicos, ciclo de vida (<i>lifecycle</i>), revisión, archivos, respuesta de la institución, generación de PDF, plantillas y biblioteca de imágenes.
<code>notificaciones</code>	Envío de correo (notificaciones de creación y de cambio de estado).
<code>mentores</code>	Catálogo de mentores, alta manual e importación masiva por CSV.
<code>reportes</code>	Tablero (<i>dashboard</i>) de métricas y exportaciones a CSV y PDF.

Capítulo 4

Estructura del proyecto

El repositorio separa la infraestructura (raíz) del código de la aplicación (app/). La raíz solo contiene Dockerfile, los docker-compose.*.yaml, el Makefile, .env.example y configuración. Todo el código nuevo vive dentro de app/.

4.1. Árbol de la raíz del repositorio

```
solicitudes/
|-- Dockerfile                # multi-stage; dependencias de WeasyPrint incluidas
|-- docker-compose.dev.yaml   # nginx + web + db + mailhog
|-- docker-compose.test.yaml  # postgres-test (sin contenedor de app)
|-- Makefile                  # atajos sobre docker compose exec web
|-- .env.example              # plantilla de variables de entorno
|-- nginx/                    # configuracion de nginx (TLS dev)
|-- certs/                    # certificados autofirmados de desarrollo
|-- app/                      # raíz del proyecto Django (montada en /app)
|-- acceptance/               # pruebas de aceptacion (behave + selenium)
|-- srs/                      # SRS formal (LaTeX)
'-- CLAUDE.md                 # reglas de arquitectura y flujo de trabajo
```

4.2. Árbol de app/

```
app/
|-- manage.py
|-- pyproject.toml
|-- requirements.txt          # dependencias de runtime
|-- requirements-dev.txt      # dependencias de desarrollo y pruebas
|-- config/                  # settings + URLs raíz
|   |-- settings/            # base.py, dev.py, prod.py, test_postgres.py
|   |-- urls.py
|   |-- wsgi.py
|   '--- asgi.py
|-- _shared/                  # excepciones, middleware, auth, auditoria
|-- usuarios/                 # JWT, roles, perfil, directorio
|-- solicitudes/              # nucleo (tipos, formularios, lifecycle, PDF...)
|-- notificaciones/           # envio de correo
|-- mentores/                 # catalogo de mentores
|-- reportes/                 # dashboard + exportaciones
|-- templates/                # base.html + components/ + por-app
|-- static/                   # CSS, JS, imagenes
```

```
|-- media/                # archivos subidos (gitignored)
|-- locale/              # i18n (es_MX)
'-- tests-e2e/           # flujos de navegador (Playwright)
```

4.3. Estructura interna de una app por capas

Cada app de dominio replica la misma estructura por capas. Por ejemplo, `usuarios/` contiene:

```
usuarios/
|-- views/                # capa de presentacion (HTTP)
|-- services/            # logica de negocio
|-- repositories/        # ORM contenido aqui
|-- models/              # modelos del ORM de Django
|-- schemas.py           # DTOs de Pydantic
|-- exceptions.py        # subclases de AppError
|-- dependencies.py      # cableado de dependencias (composicion)
|-- permissions.py       # reglas de autorizacion por rol
|-- middleware.py        # validacion de JWT
|-- migrations/          # migraciones de base de datos
|-- seeders.py           # datos sembrados de desarrollo
'-- tests/               # pruebas unitarias y de integracion
```

El núcleo `solicitudes/` agrupa además submódulos por característica: `tipos/` (catálogo de tipos de solicitud), `formularios/` (formularios dinámicos), `intake/` (alta), `lifecycle/` (máquina de estados), `revision/` (cola y atención del personal), `archivos/` (adjuntos), `respuesta/` (respuesta de la institución), `pdf/` (generación con WeasyPrint), `plantilla_assets/` (biblioteca de imágenes) y `plantillas` (editor de plantillas).

Capítulo 5

Instalación y despliegue

El stack de desarrollo se levanta por completo en Docker: **nginx** (terminación TLS en el puerto 443), **web** (Django), **db** (PostgreSQL) y **mailhog** (captura de SMTP en el puerto 8025).

5.1. Requisitos previos

- Docker Desktop (o Docker Engine 24+ con Compose v2).
- git.

Los certificados TLS de desarrollo para `localhost` ya están incluidos en el repositorio; no es necesario generarlos.

5.2. Instalación paso a paso

5.2.1. 1. Levantar el stack

```
docker compose -f docker-compose.dev.yml up -d --build
```

5.2.2. 2. Aplicar migraciones

```
docker compose -f docker-compose.dev.yml exec -T web python manage.py migrate
```

5.2.3. 3. Sembrar datos de desarrollo

```
docker compose -f docker-compose.dev.yml exec -T web python manage.py seed
```

El sembrado es **idempotente**: volver a ejecutarlo preserva las filas agregadas a mano. Para reconstruir desde cero, use **-fresh**:

```
docker compose -f docker-compose.dev.yml exec -T web python manage.py seed --fresh
```

Para ejecutar el sembrador de una sola app:

```
docker compose -f docker-compose.dev.yml exec -T web python manage.py seed --only usuarios
```

5.2.4. 4. Acceder al sistema

Abra <https://localhost/auth/dev-login> y haga clic en cualquier rol para iniciar sesión. El navegador puede advertir sobre el certificado autofirmado; acéptelo una vez.

5.3. URLs disponibles (tras levantar y sembrar)

URL	Descripción
https://localhost/auth/dev-login	Selector de login de desarrollo (solo con DEBUG ; será reemplazado por el SSO institucional).
https://localhost/auth/me	Perfil del usuario actual.
https://localhost/solicitudes/admin/ tipos/	Catálogo de tipos de solicitud (solo admin).
https://localhost/solicitudes/admin/ tipos/ nuevo/ tipo/	Crear nuevo tipo.
https://localhost/solicitudes/admin/ tipos/ plantillas/	Catálogo de plantillas de PDF.
https://localhost/solicitudes/admin/ tipos/ plantillas/ assets/	Biblioteca de assets para plantillas.
http://localhost:8025/	Mailhog (correo saliente capturado).

5.4. Usuarios sembrados

`python manage.py seed` crea un usuario por rol, con la matrícula que utiliza el selector de `/auth/dev-login`:

Rol	Matrícula	Correo
ALUMNO	ALUMNO_TEST	alumno.test@uaz.edu.mx
DOCENTE	DOCENTE_TEST	docente.test@uaz.edu.mx
CONTROL_ESCOLAR	CE_TEST	ce.test@uaz.edu.mx
RESPONSABLE_PROGRAMA	RP_TEST	rp.test@uaz.edu.mx
ADMIN	ADMIN_TEST	admin.test@uaz.edu.mx

5.5. Servicios del stack

Nginx con TLS. El servicio `nginx-dev` termina TLS en `:443` (y expone `:80`), usando los certificados autofirmados de `certs/` montados en modo solo lectura. Hace de *proxy* hacia el contenedor `web` (Django en `:8000`) y hacia `mailhog`.

Mailhog. Captura todo el correo saliente en desarrollo. Su bandeja se consulta en <http://localhost:8025/>; es útil para inspeccionar las notificaciones que dispara el sistema al crear o transicionar solicitudes.

Base de datos. El servicio `db` corre PostgreSQL con un *healthcheck* (`pg_isready`); el contenedor `web` espera a que esté sano antes de arrancar. Los datos persisten en el volumen `solicitudes_dev_data`.

Web. El contenedor `web` arranca el binario standalone de Tailwind en modo *watch* y el servidor de desarrollo de Django de forma simultánea, montando `./app` como volumen para recarga en caliente.

Capítulo 6

Comandos de operación

El `Makefile` ofrece atajos; cada uno tiene su equivalente crudo de `docker compose`, de modo que `make` es opcional. En las equivalencias, `<DC>` abrevia `docker compose -f docker-compose.dev.yml`.

Tarea	make	Equivalente docker compose
Levantar el stack	<code>make up</code>	<code><DC> up -d -build</code>
Detener el stack	<code>make down</code>	<code><DC> down</code>
Reconstruir imagen web	<code>make build</code>	<code><DC> build web</code>
Ver logs de web	<code>make logs</code>	<code><DC> logs -f web</code>
Shell de Django	<code>make shell</code>	<code><DC> exec web python manage.py shell</code>
Aplicar migraciones	<code>make migrate</code>	<code><DC> exec -T web python manage.py migrate</code>
Generar migraciones	<code>make makemigrations</code>	<code><DC> exec -T web python manage.py makemigrations</code>
Sembrar datos (idempotente)	<code>make seed</code>	<code><DC> exec -T web python manage.py seed</code>
Sembrar datos (recrear)	<code>make seed-fresh</code>	<code><DC> exec -T web python manage.py seed -fresh</code>
Lint (ruff)	<code>make lint</code>	<code><DC> exec -T web ruff check .</code>
Type-check (mypy)	<code>make type</code>	<code><DC> exec -T web mypy .</code>
Pruebas (unit+integración)	<code>make test</code>	<code><DC> exec -T web pytest</code>
Cobertura (terminal)	<code>make coverage</code>	<code><DC> exec -T web pytest -m "not e2e-cov" -cov-report=term-missing</code>
Cobertura (HTML)	<code>make coverage-html</code>	<code><DC> exec -T web pytest -m "not e2e-cov" -cov-report=html</code>
Build CSS (una vez)	<code>make css</code>	<code><DC> exec -T web tailwindcss -i /app/static/css/app.css -o /app/static/css/app.build.css -minify</code>
Build CSS (watch)	<code>make css-watch</code>	<code><DC> exec -T web tailwindcss ... -watch=always</code>
Detener + borrar volúmenes	<code>make clean</code>	<code><DC> down -v</code>

Ayuda. `make help` lista todos los objetivos disponibles con su descripción.

Capítulo 7

Configuración

La configuración se controla por variables de entorno. El archivo `.env.example` es la plantilla de referencia. En desarrollo, los valores se inyectan directamente en `docker-compose.dev.yml`; en producción se proveen mediante un `.env` o el orquestador.

Variable	Descripción
DJANGO_SETTINGS_MODULE	Módulo de settings activo (<code>config.settings.dev</code> o <code>...prod</code>).
SECRET_KEY	Clave secreta de Django. Cambiar en producción.
ALLOWED_HOSTS	Lista de hosts permitidos, separada por comas.
Base de datos (producción)	
DB_HOST	Host de PostgreSQL.
DB_NAME	Nombre de la base.
DB_USER	Usuario.
DB_PASSWORD	Contraseña.
DB_PORT	Puerto (por defecto 5432).
JWT / autenticación	
JWT_SECRET	Secreto para validar los JWT.
JWT_ALGORITHM	Algoritmo (por defecto HS256).
AUTH_PROVIDER_LOGIN_URL	URL de login del proveedor externo.
AUTH_PROVIDER_LOGOUT_URL	URL de logout del proveedor externo.
SIGA (proveedor de identidad)	
SIGA_BASE_URL	URL base del servicio SIGA.
SIGA_TIMEOUT_SECONDS	Timeout en segundos (por defecto 5).
SMTP (correo, producción)	
EMAIL_HOST	Host SMTP.
EMAIL_PORT	Puerto (por defecto 587).
EMAIL_HOST_USER	Usuario SMTP.
EMAIL_HOST_PASSWORD	Contraseña SMTP.
EMAIL_USE_TLS	Usar TLS (<code>true/false</code>).
EMAIL_TIMEOUT	Timeout en segundos (por defecto 10).
DEFAULT_FROM_EMAIL	Remitente por defecto (<code>no-reply@uaz.edu.mx</code>).
Sitio	
SITE_BASE_URL	URL base del sitio (<code>https://localhost</code> en dev).

Valores de desarrollo. En `docker-compose.dev.yml` el contenedor `web` usa `DB_HOST=db`, `DB_NAME/USER/PASSWORD=solicitudes`, `EMAIL_HOST=mailhog` y `EMAIL_PORT=1025`, entre otros.

Capítulo 8

Pruebas

El proyecto cuenta con varios niveles de prueba, cada uno con su propio comando.

8.1. Pruebas unitarias y de integración

Se ejecutan con `pytest` dentro del contenedor `web` (en proceso, sobre SQLite):

```
make test
# equivalente:
docker compose -f docker-compose.dev.yml exec -T web pytest
```

Las pruebas se etiquetan con *markers*. Para excluir las de navegador se usa `-m "not e2e"`; para ejecutar solo las de navegador, `-m e2e`. La suite de extremo a extremo en proceso usa Playwright y requiere un arranque único de Chromium dentro del contenedor:

```
make e2e-install      # instala Chromium + dependencias (una vez)
make e2e              # corre la suite Playwright (Tier 1 + Tier 2)
make e2e-headed       # con navegador visible (--headed --slowmo 200)
```

También puede ejecutarse contra un PostgreSQL efímero con `make e2e-postgres`.

8.2. Cobertura

```
make coverage          # reporte en terminal con líneas faltantes
make coverage-html     # reporte HTML en app/htmlcov/
```

Pasando `E2E=1` (por ejemplo `make coverage E2E=1`) se añade la suite Playwright a la medición. La cobertura global medida del proyecto es del **94%** sobre 5515 sentencias.

8.3. Pruebas de aceptación (behave + Selenium)

Las pruebas de aceptación de extremo a extremo están escritas en **Gherkin (español)** con **behave** y se ejecutan con **Selenium** sobre Google Chrome contra el stack en ejecución. Viven en la carpeta `acceptance/` y cubren una historia por módulo del equipo:

Feature	Módulo	Historia
<code>acceso.feature</code>	usuarios	Inicio de sesión por rol / bloqueo de anónimos.

Feature	Módulo	Historia
<code>intake.feature</code>	<code>solicitudes/intake</code>	Alta de solicitudes desde el alumno.
<code>archivos.feature</code>	<code>solicitudes/archivos</code>	Adjuntar y descargar archivos (con autorización).
<code>revision.feature</code>	<code>solicitudes/revision</code>	Cola de revisión y atención del personal.
<code>plantillas.feature</code>	<code>solicitudes/pdf</code>	Catálogo y editor de plantillas PDF.
<code>mentores.feature</code>	<code>mentores</code>	Importación CSV de mentores.
<code>reportes.feature</code>	<code>reportes</code>	Dashboard administrativo y export CSV.

Requisitos. El stack debe estar levantado y sembrado (`make up && make seed`); la suite ataca `https://localhost` (el *driver* ignora el certificado autofirmado con `acceptInsecureCerts`). Google Chrome debe estar instalado (Selenium Manager descarga el *driver*). Las dependencias se instalan en un *virtualenv* aparte del proyecto Django:

```
cd acceptance
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Ejecución.

```
cd acceptance
.venv/bin/behave                    # toda la suite
.venv/bin/behave features/intake.feature # una sola feature
.venv/bin/behave -D headless=false      # ver el navegador
.venv/bin/behave -D base_url=https://localhost # apuntar a otro entorno
```

Las aserciones se hacen siempre sobre lo que **ve el usuario** en el navegador (textos, URLs, páginas de error), nunca sobre códigos HTTP crudos.

8.4. Pruebas de rendimiento (JMeter)

Las pruebas de carga se ejecutan con Apache JMeter a partir del plan de prueba `solicitudes_performance` que ejercita los flujos críticos (login, alta de solicitud, cola de revisión, descarga de PDF) contra el stack en ejecución. Para correrlas sin interfaz gráfica:

```
jmeter -n -t solicitudes_performance.jmx -l resultados.jtl -e -o reporte_html
```

El reporte HTML resultante (`reporte_html/`) muestra latencias, *throughput* y porcentaje de error por petición.

8.5. Complejidad ciclomática y mantenibilidad (radon)

La complejidad ciclomática y el índice de mantenibilidad se miden con **radon**:

```
radon cc app/ -s -a      # complejidad ciclomatica (con promedio)
radon mi app/            # indice de mantenibilidad
```

El código se mantiene en rango **A** de mantenibilidad en prácticamente todos los módulos, y la complejidad ciclomática por función es baja (rango A en la gran mayoría).

8.6. Estilo (PEP 8) y tipado

El cumplimiento de estilo se verifica con **ruff** y el tipado estático con **mypy** en modo estricto:

```
make lint    # ruff check .
make type    # mypy .
```

Capítulo 9

Mantenimiento y extensión

9.1. Añadir una nueva característica respetando las capas

Toda característica nueva debe respetar el flujo **Vista** → **Servicio** → **Repositorio** y usar DTOs de Pydantic en las fronteras. El procedimiento recomendado es:

1. **Modelo y migración.** Defina o modifique el modelo del ORM en `models/` y genere la migración con `make makemigrations`, luego `make migrate`.
2. **DTOs.** Declare los esquemas de Pydantic en `schemas.py` para la entrada y la salida del servicio.
3. **Repositorio.** Implemente el acceso a datos en `repositories/`, manteniendo el ORM contenido aquí; traduzca entre modelos y DTOs.
4. **Servicio.** Codifique la lógica de negocio en `services/`, operando solo sobre DTOs (sin HTTP ni ORM).
5. **Excepciones.** Defina las excepciones de dominio en `exceptions.py` como subclases de `AppError`, con su `user_message` en español.
6. **Vista y plantilla.** Implemente la vista en `views/`, validando la entrada con un formulario de Django y renderizando la plantilla a partir de los DTOs.
7. **Cableado.** Registre las dependencias en `dependencies.py` y las rutas en `urls.py`.
8. **Pruebas.** Agregue pruebas en `tests/` y verifique con `make test` y `make coverage`.

Añadir una app completa. Cree la carpeta bajo `app/` replicando la estructura por capas, regístrela en `INSTALLED_APPS` (`config/settings/base.py`) e incluya sus `urls.py` en `config/urls.py`.

9.2. Datos sembrados (seeders)

Para sembrar datos de una app nueva, agregue un `seeders.py` en la raíz de la app con una función `run(*, fresh: bool) -> None`. El comando `manage.py seed` lo descubre automáticamente:

```
# app/<su_app>/seeders.py
DEPENDS_ON: list[str] = ["usuarios"] # opcional: corre despues de estas apps

def run(*, fresh: bool) -> None:
    if fresh:
        ... # borra las filas que este seeder posee
        ... # update_or_create de sus filas de muestra
```

Use `update_or_create` para mantener la idempotencia y borre filas solo cuando `fresh` es verdadero.

9.3. Migraciones

Genere migraciones tras cualquier cambio de modelo con `make makemigrations` y aplíquelas con `make migrate`. Las migraciones se versionan junto con el código en la carpeta `migrations/` de cada app.

Capítulo 10

Solución de problemas

10.1. Advertencia de certificado autofirmado

El *stack* de desarrollo usa nginx con un certificado autofirmado para `localhost`. El navegador advertirá que la conexión no es de confianza la primera vez; acepte la excepción una sola vez. Para las pruebas de aceptación, el *driver* de Selenium ya ignora el certificado mediante `acceptInsecureCerts`.

10.2. Puertos ocupados

El *stack* expone los puertos 80, 443, 5432 y 8025 en el anfitrión. Si alguno está ocupado, el arranque fallará. Detenga el proceso que lo usa o ajuste el mapeo de puertos en `docker-compose.dev.yml`. Para verificar qué proceso ocupa un puerto:

```
lsof -i :443
```

10.3. Reconstruir la imagen

Si cambian las dependencias (`requirements.txt` / `requirements-dev.txt`) o el `Dockerfile`, reconstruya la imagen:

```
make build
# o forzando una reconstruccion completa:
docker compose -f docker-compose.dev.yml build --no-cache web
```

10.4. Estado inconsistente o base corrupta en desarrollo

Para partir de cero (detiene todo y borra los volúmenes, incluida la base de datos) y volver a levantar:

```
make clean
make up
make migrate
make seed
```

10.5. Chromium ausente en pruebas de navegador

Si las pruebas Playwright fallan por falta de navegador, ejecute el arranque único:

```
make e2e-install
```